

JomCode Coding Bootcamp

Testing with pytest

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Write assertions for normal, boundary and error cases.
- 02 Use parametrization and fixtures without hiding test behavior.
- 03 Use a failing test to guide a specific repair.

Plain meaning → technical view → use

PLAIN MEANING

Check that code still behaves as intended after a change.

TECHNICAL VIEW

pytest assertions, fixtures and parametrization exercise normal and boundary cases.

WHEN YOU USE IT

Catch a duplicate-registration regression before deploying.

TIP

Name a test after the behavior; run it before and after a deliberate small change.

Install pytest in your virtual environment

TERMINAL >_ PowerShell / zsh

```
> python -m pip install pytest  
> python -m pytest --version
```

Windows may use py; macOS may use python3. Prefer the project .venv Python.

Worked example 1: Simple assertion



chapter_13_testing_with_pytest.py

```
1 def remaining(capacity, registered):
2     return capacity - registered
3
4 def test_remaining():
5     assert remaining(20, 8) == 12
6     assert remaining(5, 5) == 0
```

PREDICT Which state is observable to the caller?

Example 1: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
pytest reports the test passes. A wrong return produces a meaningful assertion failure.
```

WHY THIS HAPPENS

The return value is the contract, so the test does not care about internal variable names.

Exercise 1: Full capacity matrix

BUILD

Write `validate_capacity` and parametrized tests for valid and invalid values.

CHECK

Valid 1,500. Invalid 0,501,True,"20". All error cases raise `ValueError`.

IF STUCK

Assert both acceptance and rejection outcomes.

Worked example 2: Parametrized boundaries



chapter_13_testing_with_pytest.py

```
1 import pytest
2
3 @pytest.mark.parametrize("value", [1, 500])
4 def test_valid_capacity(value):
5     assert type(value) is int and 1 ≤ value ≤ 500
```

PREDICT How many tests does this definition create?

Example 2: output and reason

TERMINAL >_ EXPECTED OUTPUT

Two collected cases pass.

WHY THIS HAPPENS

Parametrization expands one test function into a test per input.

Exercise 2: Empty-list regression

BUILD

Write a `total_capacity` function and tests for empty and nonempty data.

CHECK

`[]` $\Rightarrow$ 0. Capacities 3 and 7 $\Rightarrow$ 10.

IF STUCK

Use the empty input as a named regression case.

Worked example 3: Isolated fixture



chapter_13_testing_with_pytest.py

```
1 import pytest
2
3 @pytest.fixture
4 def record():
5     return {"title": "Python", "capacity": 20}
6
7 def test_record_capacity(record):
8     assert record["capacity"] == 20
```

PREDICT Should a fixture erase a student's Atlas database?

Example 3: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
The fixture supplies fresh data to the test.
```

WHY THIS HAPPENS

Never. Destructive fixtures in this package require explicitly guarded private local test databases.

Exercise 3: Normalization regression

BUILD

Test that trimming a name preserves its meaningful internal double space.

CHECK

" Ada L. " => "Ada L.". A split/join implementation fails.

IF STUCK

Write the exact expected string.

Checkpoint

PROVE IT

A test suite catches invalid boundaries and protects a previously repaired behavior.